

运维工程师 · 二面题库

南京华苏科技 / 中移互联网
(通俗易懂版 · 17 道题)

☑ 本版针对二面（技术主管/项目经理面），聚焦 JD 6 大职责：
上线流程、配置变更、DB 变更、紧急变更、CI/CD、K8s 发布回滚、场景题。
一面考基础（nginx/http/抓包/排障），二面考流程意识 + 实战经验 + 风险把控。

二面核心策略：

- ① 每个答案都要带具体数字（错误率、用户数、恢复时间）；
- ② 突出个人决策（「我判断回滚」而不是「我们团队回滚」）；
- ③ 强调风险意识（回滚预案、变更审核、灰度发布）；
- ④ 主动提流程优化（自动化、卡点、复盘）。

一、上线流程与变更管理（5 道）

☑☑ 本方向 5 道题，紧扣 JD 对应职责。

☑ [1-1] JD 职责1：上线流程管理

一面没问流程，二面必问。JD 第一条就是上线流程。

☑ 你们公司完整的上线流程是怎样的？灰度发布怎么做？回滚预案怎么写？

☑ 人话解释：把上线想象成火车发车：

- ① 发车前安检（需求评审）→ ② 试车（测试环境验证）→ ③ 申请发车（上线申请审核）→ ④ 进站台（上线窗口）→ ⑤ 慢慢开（灰度发布）→ ⑥ 到站检查（上线后巡检）→ ⑦ 出事了倒车（回滚）

灰度发布三种方式：

- 按流量比例：Nginx/Ingress weight 5%→50%→100%
- 按用户：白名单（内部员工、特定用户 ID）
- 按机房/区域：先上一个机房或城市

回滚预案三件套：

- ① 应用：Docker 镜像保留上一个 tag，kubectl rollout undo
- ② 配置：Git 管理配置文件，变更前 commit，随时回退
- ③ 数据库：DDL 前 mysqldump 备份 + 提前写好回滚 SQL

☑ 面试作答：完整上线流程 7 步：需求评审 → 测试环境验证 → 上线申请审批 → 上线窗口约定 → 灰度发布 → 上线后巡检 → 回滚预案就绪。

灰度发布：按比例/白名单/分区域三种，核心是小步快跑、可观测、可回滚。

回滚预案：

- 应用层：Docker 镜像 tag 保留上一版本，K8s kubectl rollout undo；
- 配置层：Git 版本管理，git checkout 旧版本；
- 数据库层：DDL 前 mysqldump 全备 + 回滚脚本。

回滚触发条件：错误率 > 5%、CPU/内存 > 80%、核心接口 P99 > 3s。

☒☒ 速记口诀：上线 7 步：评审→测试→审批→窗口→灰度→巡检→回滚。灰度三方式：比例/白名单/区域。回滚三件套：镜像/配置/数据库。

☒ [1-2] JD 职责2：配置变更管理

JD 明确写「配置变更可追溯、可回退」。

☒ 配置变更怎么做到可追溯、可回退？用什么工具？

☒ 人话解释：想象你家装修改水电：

- 可追溯：每次改前拍照（Git commit），谁改的、改了啥、什么时候全记录；
- 可回退：改坏了恢复成上一张照片（git checkout/revert）；
- 审核：改前要设计师签字（MR/Code Review）；
- 环境隔离：开发/测试/生产的水电图分开（dev/test/prod 分支）。

实际工具链：

- 配置文件放 GitLab 仓库，每次改走 MR
- 配置中心（Apollo/Nacos），每次发布有版本号可一键回滚
- 变更单 + 审批流（钉钉/飞书）
- 操作日志留痕（堡垒机录屏）

☒ 面试作答：配置变更管理四要素：

- ① 版本化：配置文件放 GitLab，分支隔离 dev/test/prod；
- ② 审核：变更走 MR，至少 1 人 Code Review；
- ③ 可回退：配置中心（Apollo/Nacos）版本号管理，一键回滚；
- ④ 留痕：变更单（审批流）+ 操作日志（堡垒机录屏）。

工具链：GitLab + Apollo/Nacos + Jenkins + 堡垒机。

☒☒ 速记口诀：配置管理四件事：Git 版本化、MR 审核、配置中心回滚、堡垒机留痕。记成「审+录+回+痕」。

☒ [1-3] JD 职责4：紧急变更处理

JD 专门有一条讲紧急变更。

☒ 紧急变更和常规变更的流程区别？紧急变更怎么快速处理？

☒ 人话解释：想象急诊 vs 门诊：

- 常规变更（门诊）：提前预约（变更单）→ 排队等号（审批）→ 到点做手术（上线窗口）
- 紧急变更（急诊）：直接进抢救室（跳过预约）→ 主治医生签字（简化审批）→

立刻开刀（马上执行）→ 事后补病历（事后补变更单+复盘）

紧急变更快速通道 4 步：

- ① 快速评估：影响范围多大？不改会不会更糟？
- ② 简化审批：口头/微信群找技术总监 + 业务负责人双签，留截图
- ③ 立即执行：边操作边群里同步进度
- ④ 事后补齐：24 小时内补变更单 + 写复盘文档

☒ 面试作答：常规 vs 紧急变更：

| 维度 | 常规变更 | 紧急变更 |

| ----- | ----- | ----- |

| 提前量 | 提前 1-3 天 | 即时 |

| 审批 | 完整审批流 | 简化（总监+业务口头同意+留证） |

| 执行窗口 | 约定低峰期 | 立即执行 |

| 回滚预案 | 提前写 | 边执行边准备 |

| 事后 | 巡检报告 | 24h 内补变更单+复盘 |

紧急变更 4 步：评估影响 → 简化审批（口头双签+截图）→ 立即执行（群内同步）→ 事后复盘。

关键原则：紧急变更不是跳过所有流程，只是压缩审批时间，操作本身必须按规范（备份、验证、回滚）。

☒☒ 速记口诀：紧急变更 4

字诀：评（评估）、批（简化审批）、做（立即执行）、补（事后复盘）。核心是留证据+可回滚。

☒ [1-4] JD 职责6：上线质量保障

JD 明确写「建立上线 checklist 和自检验证标准」。

☒ 上线前的 checklist 有哪些项？上线后怎么验证？

☒ 人话解释：把上线 checklist 想成飞机起飞前的检查单：

起飞前（8 项）：

- ☒ 代码 MR 已通过
- ☒ Jenkins 构建成功，镜像已推仓库
- ☒ 测试环境功能验证通过
- ☒ DB 变更脚本已 DBA 审核
- ☒ 回滚方案已准备（镜像/配置/DB）
- ☒ 监控告警已配置
- ☒ 相关人员已通知
- ☒ 上线窗口已确认

起飞后（5 项，30 分钟内）：

- ☒ 核心接口错误率 < 0.1%
- ☒ CPU/内存 < 70%

☑ DB 无新增慢查询

☑ 日志无 ERROR

☑ 客服无新增投诉

☑ 面试作答：上线前 checklist（8 项）：代码 MR 通过、Jenkins 构建成功、测试验证通过、DB 脚本已审核、回滚预案就绪、监控已接入、相关人已通知、上线窗口已确认。

上线后验证（5 项）：错误率 < 0.1%、CPU/内存 < 70%、无新增慢查询、日志无 ERROR、客服无投诉。

灰度策略：5% 观察 10min → 50% 观察 10min → 100%。任一指标异常立即回滚。

☑☑ 速记口诀：上线前 8 项：代码、镜像、测试、DB、回滚、监控、通知、窗口。上线后 5 项：错误率、CPU 内存、慢 SQL、日志、客服。

☑ [1-5] 综合：上线故障应急

二面高频场景题。

☑ 上线后 5 分钟业务突然挂了，你怎么处理？

☑ 人话解释：记住「先救业务，再查原因」八字方针：

① 立即回滚（0-5min）：别想为什么，先切回上一版本。K8s: `kubectl rollout undo`；Nginx: `weight` 切回旧版。

② 确认恢复（5-10min）：错误率回落、核心接口正常、客服无投诉。

③ 保护现场（10min 内）：保留出问题的镜像、日志、配置快照（别删）。

④ 同步信息：群里同步「已回滚，业务恢复，原因排查中」。

⑤ 定位根因（事后）：日志 → 配置 → DB → 抓包。

⑥ 写事故报告：时间线、影响、根因、整改。

△☑ 关键：不要在出问题时现场调试！每分钟都是钱，先回滚让用户能用。

☑ 面试作答：上线故障处理流程：

第一阶段（0-10min 救火）：立即回滚 → 确认业务恢复 → 保护现场；

第二阶段（10min-2h 通报）：群内同步进度 → 通知受影响方；

第三阶段（2h+ 复盘）：定位根因 → 写事故报告 → 落地整改。

核心原则：先恢复业务再查根因；回滚是第一选择不是最后选择。

☑☑ 速记口诀：上线故障八字诀：先救业务，再查原因。三步：回滚 → 确认 → 查根因。保护现场很重要，别删日志。

二、数据库变更管理（4 道）

☒☒ 本方向 4 道题，紧扣 JD 对应职责。

☒ [2-1] JD 职责3：数据库变更管理

大表 DDL 是高频坑，必问。

☒ 给一张 5000 万行的大表加字段/加索引，怎么操作才不锁表？

☒ 人话解释：把大表想象成 1000 人的大教室，要给每个人发新表（加字段）：

- 笨办法（直接 ALTER）：让所有人站起来重排座位（锁表），期间没人能上课。
- 聪明办法三种：

① MySQL 8.0 在线 DDL：ALTER TABLE t ADD COLUMN c INT, ALGORITHM=INPLACE, LOCK=NONE; 建临时表+增量追平，不锁表，用户无感。

② pt-online-schema-change (Percona)：建影子表+触发器同步+RENAME 切换，MySQL 5.x 也能用。

③ gh-ost (GitHub)：用 binlog 代替触发器，更安全，大厂主流方案。

执行时间：选业务低峰期（凌晨 2-5 点），观察主从延迟。

回滚：gh-ost 执行中随时 Ctrl+C 终止，不影响原表。

☒ 面试作答：大表 DDL 不锁表 3 种方案：

① MySQL 8.0 在线 DDL：ALGORITHM=INPLACE, LOCK=NONE，支持加列/加索引；

② pt-online-schema-change：影子表+触发器+RENAME，MySQL 5.x 可用；

③ gh-ost：binlog 代替触发器，更稳定，大厂主流。

风险评估：表大小、操作类型、执行时机、影响行数、磁盘空间、主从延迟。

执行：低峰期 + 分批 + 观察主从延迟。回滚=Ctrl+C 终止。

☒☒ 速记口诀：大表 DDL 三工具：MySQL 8.0

在线 DDL、pt-osc、gh-ost。口诀：「8 用在线、5 用 pt-osc、大厂用 gh-ost」。

☒ [2-2] JD 职责3：变更风险评估

JD 明确写「评估变更风险并制定回滚方案」。

☒ 拿到一个数据库 DDL/DML 变更申请，你怎么评估风险？怎么制定回滚方案？

☒ 人话解释：把数据库变更想象成给心脏做手术：

风险评估 4 问：

① 改的是哪张表？多大？（100 行 vs 10 亿行，天差地别）

② 改的是啥操作？（加列最安全、改类型最危险、DELETE 全表最可怕）

③ 高峰期还是低峰期执行？（低峰执行）

④ 有没有主键/索引？影响多少行？（全表扫 vs 走索引，差 100 倍）

回滚方案（按操作类型）：

- DDL（加列/加索引）：MySQL 8.0 在线 DDL 或 gh-ost，支持随时中断；
- DML（UPDATE/DELETE）：执行前先 mysqldump 备份或 SELECT 导出来，回滚=反向

UPDATE/INSERT;

- 数据迁移：先建影子表，保留旧表 7 天再删。

铁律：任何 DML 变更前必须备份！任何 DDL 变更前必须有在线 DDL 方案！

☑ 面试作答：风险评估 4 维度：表规模、操作类型、执行时机、影响行数。

回滚方案：

- DDL：在线 DDL (INPLACE+LOCK=NONE) 或 gh-ost，可中断；
- DML (UPDATE/DELETE)：执行前 SELECT 备份要修改的数据，回滚=反向 UPDATE/INSERT；
- INSERT：回滚=DELETE WHERE 主键 IN (...);
- 数据迁移：保留旧表 7-30 天。

审批红线：全表 DELETE/UPDATE 必须 DBA+技术总监双签；无回滚方案不批；无备份不执行。

☑☑ 速记口诀：DB 变更风险四问：表多大？改啥？啥时候？影响几行？回滚三原则：DDL用在线DDL、DML先备份、迁移留旧表。

☑ [2-3] JD 任职要求5: Redis

JD 要求熟悉 Redis。

☑ Redis 大 key 和热 key 怎么发现和处理？

☑ 人话解释：把 Redis 想象成超市储物柜：

大 key：某个柜子塞了一头大象 (value > 10KB 或 list/hash 上万元素)。

发现：redis-cli --bigkeys、slowlog get。

处理：

- string 大 value：拆成多个 key
- list/hash 太大：分桶 (hash_1, hash_2...)
- 删大 key：用 UNLINK 异步删除 (不阻塞)

热 key：某个柜子被疯狂抢 (QPS 过万，比如秒杀库存)。

发现：redis-cli --hotkeys (需 LFU 模式 maxmemory-policy allkeys-lfu)。

处理：

- 应用层本地缓存 (Caffeine/Guava) 挡一层
- 热 key 多分片 (hotkey_0~hotkey_9 随机读)
- 加互斥锁防缓存击穿

☑ 面试作答：大 key：

发现：redis-cli --bigkeys、slowlog。

处理：string 拆分、list/hash 分桶、删除用 UNLINK 异步。

热 key：

发现：redis-cli --hotkeys (LFU 模式)、业务监控。

处理：应用层本地缓存、热 key 多分片、互斥锁防击穿、限流降级。

预防：设计时 value < 10KB；热点数据加本地缓存。

☒☒ 速记口诀：大 key 发现用 --bigkeys，拆分+UNLINK 异步删；热 key 发现用 --hotkeys，本地缓存+多分片。

☒ [2-4] JD 任职要求5: Redis

Redis 持久化也是常考。

☒ Redis 的 RDB 和 AOF 有什么区别？生产怎么选？数据丢了怎么恢复？

☒ 人话解释：把 Redis 想成记账员：

- RDB（快照）：每隔一段时间把账本拍张照片（dump.rdb）。优点：文件小、恢复快；缺点：两次快照间数据可能丢。
- AOF（追加日志）：每改一次账就记一条流水（appendonly.aof）。优点：数据最安全（最多丢 1 秒）；缺点：文件大、恢复慢。

生产怎么选？两个都开（混合持久化）：

Redis 4.0+ 支持 aof-use-rdb-preamble yes：AOF 文件前半是 RDB 快照（恢复快），后半是 AOF 增量（数据安全）。

数据丢了恢复：

① 停 Redis → 把备份的 dump.rdb 或 appendonly.aof 放回 data 目录 → 启动

② AOF 损坏：redis-check-aof --fix appendonly.aof

☒ 面试作答：RDB vs AOF：

| 特性 | RDB | AOF |

| ----- | ----- | ----- |

| 方式 | 定时快照 | 追加写日志 |

| 数据安全 | 可能丢最后 N 分钟 | 最多丢 1 秒 |

| 文件大小 | 小 | 大 |

| 恢复速度 | 快 | 慢 |

生产方案：混合持久化（aof-use-rdb-preamble yes）——AOF 前半 RDB+后半增量，既快又安全。

数据恢复：停 Redis → 替换 data 目录文件 → 启动；AOF 损坏用 redis-check-aof --fix。

☒☒ 速记口诀：RDB=快照（快但可能丢）、AOF=日志（慢但安全）。生产用混合持久化。恢复：停→替换文件→启。

三、CI/CD (Jenkins + GitLab) (3 道)

☒☒ 本方向 3 道题，紧扣 JD 对应职责。

☒ [3-1] JD 任职要求2、3：GitLab CI/CD + Jenkins

JD 明确要求 GitLab CI/CD 和 Jenkins。

☒ 一条完整的 CI/CD 流水线是怎样的？Jenkins Pipeline 和 GitLab CI/CD 有啥区别？

☒ 人话解释：把 CI/CD 想象成工厂流水线：

原材料（代码）→ 质检（编译+单测）→ 打包（构建镜像）→ 入库（推镜像仓库）→
出厂（部署测试）→ 试吃（集成测试）→ 上架（生产）

完整流水线 6 阶段：

① Checkout（拉代码）→ ② Build（编译）→ ③ Test（单测+Sonar）→ ④ Package（docker build+push）→ ⑤ Deploy（kubectl apply 到测试）→ ⑥ Verify（集成测试→生产）

Jenkins vs GitLab CI/CD：

- Jenkins：老牌通用，插件多、功能全，但配置复杂、UI 老。Pipeline 用 Groovy（Jenkinsfile）。
 - GitLab CI/CD：和 GitLab 深度集成，配置简单（.gitlab-ci.yml），UI 现代。
- 选型：用 GitLab 管代码就直接用 GitLab CI/CD；复杂定制用 Jenkins。

☒ 面试作答：CI/CD 流水线 6 阶段：Checkout → Build → Test(Sonar) → Package(Docker) → Deploy(test) → Verify→Release。

Jenkins vs GitLab CI/CD：

维度	Jenkins	GitLab CI/CD
配置	Jenkinsfile (Groovy)	.gitlab-ci.yml (YAML)
集成	需配 Git 插件	原生集成 GitLab
灵活性	高	中
适用	复杂多语言/多云	用 GitLab 的团队

配套工具：SonarQube（代码质量）、Nexus/Harbor（制品仓库）、ArgoCD（GitOps 部署）。

☒☒ 速记口诀：流水线 6 步：拉代码→编译→测试→打包→部署→验证。Jenkins=Groovy，GitLab=YAML。Sonar=代码质量，Nexus/Harbor=制品仓库。

☒ [3-2] JD 任职要求2：Git

JD 要求熟悉 Git 操作。

☒ Git Flow 和 GitHub Flow 有什么区别？rebase 和 merge 冲突怎么处理？

☒ 人话解释：GitHub Flow（简单）：只有 main + feature 分支，开发完合并到 main 就发。适合小团队、持续发布。

Git Flow（复杂）：5 条分支——master（生产）、develop（开发主线）、feature（功能）、release（发布）、hotfix（线上修 bug）。适合大团队、按版本发布。

merge vs rebase:

- merge: 两条线合并成一个节点, 多一个 merge commit, 保留完整历史
- rebase: 把你的提交搬家到目标分支后面, 历史是一条直线, 干净

冲突处理:

git pull → 冲突了 → 打开文件找 <<<<<< ===== >>>>>> → 手动改 → git add → git commit

rebase 冲突: 改完 git add → git rebase --continue

⚠️❌ 不要对已经 push 的公共分支做 rebase!

❌ 面试作答: Git Flow vs GitHub Flow:

| 维度 | Git Flow | GitHub Flow |

|-----|-----|-----|

| 分支数 | 5 条 | 2 条 |

| 发布 | 按版本周期 | 随时合并随时发 |

| 适用 | 大团队 | 小团队、持续发布 |

merge vs rebase: merge 保留分叉历史; rebase 历史线性干净。

冲突处理: 找 <<<<<< ===== >>>>>> 标记 → 手动改 → git add → git commit (或 git rebase --continue)。

原则: 公共分支用 merge 保留历史; 个人分支用 rebase 保持整洁。不要对已 push 的公共分支 rebase。

❌❌ 速记口诀: Git Flow=5条分支复杂版, GitHub

Flow=2条分支简单版。merge=合并留历史, rebase=变基变直线。冲突改完 git add + commit。

❌ [3-3] JD 任职要求3: Sonar + Nexus

JD 明确列了 Sonar 和 Nexus。

❌ SonarQube 和 Nexus 在 CI/CD 里分别做什么? 怎么集成到 Jenkins?

❌ 人话解释: 把 CI/CD 想成开餐厅:

- SonarQube (质检员): 每道菜检查有没有头发 (bug)、没洗的菜 (漏洞)、重复做的菜 (代码重复率)。不合格不能上桌。
- Nexus (仓库管理员): 做好的菜 (Jar 包、Docker 镜像) 放仓库, 下次直接取, 不用重新做。分 hosted (私有)、proxy (代理缓存)、group (聚合)。

Jenkins Pipeline 集成:

```
stage('SonarQube') { sh 'mvn sonar:sonar -Dsonar.host.url=http://sonar:9000' }
```

```
stage('Publish') { sh 'mvn deploy' } // 推到 Nexus
```

❌ 面试作答: SonarQube: 静态代码扫描 (bug、漏洞、坏味道、重复率、覆盖率), 质量门禁不达标则流水线失败。

Nexus: 制品仓库 (Jar、War、Docker 镜像、Npm 包), 三种类型

hosted/proxy/group，加速构建+版本管理。

Jenkins 集成：

- SonarQube：装 SonarQube Scanner 插件，stage 执行 mvn sonar:sonar；
- Nexus：配置 settings.xml 的 distributionManagement，mvn deploy 上传；
- Docker：docker build → docker push nexus.example.com/repo/image:tag。



速记口诀：Sonar=代码质检员（bug/漏洞/重复率），Nexus=制品仓库（存包+加速构建）。集成：mvn sonar:sonar + mvn deploy。

四、K8s 发布与回滚（3 道）

☒☒ 本方向 3 道题，紧扣 JD 对应职责。

☒ [4-1] JD 任职要求4：K8s 部署/回滚

JD 要求 K8s 基本操作。

☒ K8s 里滚动更新、蓝绿发布、金丝雀发布有什么区别？

☒ 人话解释：把发布想象成换路灯：

- 滚动更新：一盏一盏换。K8s 默认方式。缺点：新旧混跑期间可能有兼容问题。
- 蓝绿发布：旁边建一套全新路灯（绿环境），全部换好测试没问题，把电闸一拉（流量切换），瞬间从旧变新。优点：切换快、无混跑；缺点：需 2 倍资源。实现：两套 Deployment + Service 切 selector。
- 金丝雀发布：先让 5%

用户用新版本，观察没问题再全量。优点：风险最小；缺点：需流量控制。实现：Ingress weight / Istio / Nginx 分流。

怎么选：小改动→滚动；大改动→蓝绿；高风险→金丝雀。

命令：kubect set image deployment/nginx nginx=nginx:1.25

kubectl rollout status/history/undo deployment/nginx

☒ 面试作答：三种发布策略：

| 策略 | 原理 | 优点 | 缺点 | 实现 |

|-----|-----|-----|-----|-----|

| 滚动更新 | 逐个替换 Pod | 省资源 | 新旧混跑兼容风险 | Deployment 默认 |

| 蓝绿发布 | 建全新环境再切流量 | 切换快 | 需 2 倍资源 | 两套 Deployment+Service |

| 金丝雀 | 小流量先验证 | 风险最小 | 需流量控制 | Ingress weight/Istio |

K8s 命令：kubectl set image、kubectl rollout status/history/undo。

☒☒ 速记口诀：滚动=逐个换（省资源），蓝绿=全建好再切电闸（快但费资源），金丝雀=先放5%（最安全）。回滚=kubectl rollout undo。

☒ [4-2] JD 任职要求4：配置变更和回滚

ConfigMap/Secret 热更新常考。

☒ K8s 里怎么更新 ConfigMap/Secret？更新后 Pod

需要重启吗？怎么不重启就生效？

☒ 人话解释：把 ConfigMap 想象成贴在容器墙上的便签纸：

- 环境变量方式挂载（envFrom）：便签抄进容器脑子里——改了便签，脑子里的不变，必须重启容器。
- Volume 挂载方式（volumeMounts）：便签贴在墙上——改了便签，墙上自动刷新（约 60 秒延迟），容器不用重启，但应用得自己重新读文件（很多应用启动时读一次，改了不重读也白搭）。

不重启生效的方案：

- ① 应用支持热加载（Nginx -s reload、Spring Cloud Bus、Apollo/Nacos 推送）
- ② 用 Reloader（开源控制器）：监听 ConfigMap 变化，自动滚动重启 Deployment
- ③ Sidecar 容器：监听配置文件变化，通知主进程

☒ 面试作答：ConfigMap/Secret 更新：kubectl edit cm xxx 或 kubectl apply -f cm.yaml。

Pod 是否重启取决于挂载方式：

- env/envFrom 挂载：必须重启（环境变量启动时注入）；
- volumeMounts 挂载：自动更新文件（约 60s），无需重启，但应用需自己重读。

不重启生效：应用热加载（Nginx reload/Spring refresh/Apollo 推送）、Reloader 控制器、Sidecar。

回滚 ConfigMap：kubectl apply -f old-configmap.yaml（需自己在 Git 保留旧版本）。

☒☒ 速记口诀：环境变量挂载=必须重启，Volume挂载=自动更新但应用要自己重读。热更新三招：应用 reload、Reloader、Sidecar。

☒ [4-3] JD 任职要求4+ 综合排障

K8s 发布失败排查高频题。

☒ K8s 发布新版本后 Pod 一直起不来（CrashLoopBackOff），你怎么排查？

☒ 人话解释：按从外到内、从容器到镜像的顺序查：

① kubectl describe pod（看 Events 定位大类）：

- ImagePullBackOff：镜像拉不下来（地址错/没登录/标签不存在）
- OOMKilled：内存不够，容器被杀
- CrashLoopBackOff：进程启动后退出（看日志）
- CreateContainerConfigError：ConfigMap/Secret 不存在

② kubectl logs --previous（看上一次崩溃日志，最有用！）

③ kubectl exec -it（进容器手动跑启动命令）

④ kubectl describe node / get events -A（节点资源/集群事件）

☒ 面试作答：K8s Pod 启动失败排查 4 步：

Step 1: kubectl describe pod 看 Events（ImagePullBackOff/OOMKilled/CrashLoopBackOff）；

Step 2: kubectl logs --previous 看上一次崩溃日志（最关键）；

Step 3: kubectl exec 进容器手动执行启动命令复现；

Step 4: kubectl describe node / get events -A 看节点资源。

常见根因：配置错误、依赖服务没起、端口冲突、镜像 tag 写错、OOM。

☒☒ 速记口诀：排查四步：describe看事件→logs看崩溃日志→exec进容器复现→describe node看资源。logs --previous 是关键！

五、综合场景与软素质（3 道）

☒☒ 本方向 3 道题，紧扣 JD 对应职责。

☒ [5-1] 综合：流程意识

二面很可能考。

☒ 研发说「就改一行配置，很简单」想跳过变更评审直接上生产，你怎么办？

☒ 人话解释：分三步：共情 → 讲风险 → 给快速方案

① 共情：「理解你赶时间，一行配置确实简单，我也想快点帮你搞定。」

② 量化风险：「但这行是改数据库连接串，万一写错，整个服务连不上 DB，恢复至少 10 分钟，影响 5000 用户。上次 XX 项目就是改一行 Redis 配置写错了 IP，挂了半小时。」

③ 给快速方案：「要不走个紧急变更——你群里 @ 总监口头同意，我 5 分钟批完，然后我们一起上，改完我帮你验证。既快又安全。」

△☒ 底线：高危操作（DB 密码、删数据、核心路由）必须走流程，不能妥协。

加分：「事后我推动把这类高频小变更做成自动化模板，审批从 1 小时降到 5 分钟。」

☒ 面试作答：沟通框架：共情 → 量化风险 → 给替代方案：

① 共情，不直接拒绝；

② 用具体例子量化风险（影响面、恢复时间、历史教训）；

③ 紧急变更快速通道（口头双签+留证+24h 补单）。

底线：高危操作必须走完整流程。

加分：主动推动流程优化（自动化模板、审批提速）。

☒☒

速记口诀：沟通三招：共情+风险量化+给方案。底线：高危操作必须走流程。加分：主动推动流程优化。

☒ [5-2] 综合：故障经验

二面必问题。

☒ 说一个你印象最深的线上故障处理案例。

☒ 人话解释：按 STAR 法则回答：

- S（背景）：什么时候、什么系统、影响多大
- T（任务）：你当时要解决什么问题
- A（行动）：你具体做了什么（重点，占 60%）
- R（结果）：最终结果 + 复盘改进

通用模板（替换成你自己的经历）：

「去年 X 月，XX 系统版本发布后核心接口错误率从 0.1% 飙到 30%，影响约 2 万用户，我是第一响应人。」

我做了 4 件事：① 立即回滚版本，3 分钟内业务恢复；② 保留现场，定位到是新版本改了 SQL 导致全表扫描；③ 协调研发加索引后重新灰度；④ 写复盘报告，推动 SQL 上线前必须 EXPLAIN

的流程卡点。

结果：故障 5 分钟内恢复，同类问题再没发生。」

△☒ 关键：突出你做了什么，用数字（错误率、用户数、恢复时间）。

☒ 面试作答：STAR 法则：

S 背景：时间+系统+影响面；

T 任务：你的角色和目标；

A 行动（重点占 60%）：先救业务（回滚）→定位根因→修复→验证；

R 结果：恢复时间+改进措施。

加分点：用数字、突出个人决策、有复盘改进、体现合作。

☒☒ 速记口诀：故障案例用

STAR：背景(S)→任务(T)→行动(A)→结果(R)。行动占60%，用数字，突出个人+复盘。

☒ [5-3] 综合：职业规划

二面主管面常问。

☒ 为什么从上家公司离职？对这个岗位（华苏/中移互联网）有什么了解？

☒ 人话解释：离职原因（别说上家坏话，说「成长不匹配」）：

「上家业务稳定，运维偏日常巡检和故障处理。我希望在 DevOps、CI/CD、发布管理方向深入实践，贵司这个岗位正好匹配我的发展方向。」

对岗位了解（结合 JD 说）：

「JD 核心是 6 块：上线流程、配置变更、DB

变更、紧急变更、敏捷发布、质量保障。本质是安全前提下提升发布效率。工具链 Jenkins/GitLab CI/CD + K8s + MySQL/Redis。中移互联网是中国移动旗下，移动业务经验是加分项，我之前有 XX 项目经验。」

△☒ 雷区：别说上家坏话；别泛泛而谈；别说「我来学习的」。

☒ 面试作答：离职原因（安全模板）：上家业务稳定，运维偏日常巡检和故障处理，希望在 DevOps/CI/CD/发布管理方向深入，贵司岗位匹配。

对岗位了解：JD 核心 6 块——上线流程、配置变更、DB 变更、紧急变更、敏捷发布、质量保障。本质是安全前提下提升发布效率。中移=中国移动旗下，移动业务经验加分。

禁忌：不说上家坏话；不泛泛而谈；不说「我来学习的」。

☒☒ 速记口诀：离职=成长方向不匹配。了解岗位=背JD核心+匹配度。中移=提移动业务经验加分。

☑ 二面临场 Checklist（进考场前默念）

- ☑ 自我介绍：名字 + 几年运维经验 + 做过的发布/变更系统 + 为什么来华苏/中移
- ☑ 主动说你上家公司的完整上线流程（评审→灰度→回滚）
- ☑ 准备 1-2 个真实故障案例（用 STAR 法则，带数字）
- ☑ 大表 DDL 不锁表：MySQL8.0 在线 DDL / pt-osc / gh-ost
- ☑ K8s 回滚命令：kubectl rollout undo
- ☑ 紧急变更流程：评→批→做→补
- ☑ Redis 大 key：--bigkeys + UNLINK；热 key：--hotkeys + 本地缓存
- ☑ 研发想跳流程：共情+风险量化+给紧急变更方案
- ☑ 离职原因：成长方向不匹配（别说上家坏话）
- ☑ 中移互联网：主动提你了解或接触过的移动业务项目（加分）
- ☑ 反问环节：可以问「公司现在的 CI/CD 流水线用的 Jenkins 还是 GitLab？」「发布频率大概多少？」